

# CampusEvents

## *Campus Event & Ticket Booking System*

COP 4710 — Database Systems    University of South Florida    Spring 2026

---

**Final Report** — Due May 1, 2026

### Contents

---

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| 1. Team Roster .....                                                       | 2  |
| 2. System Overview .....                                                   | 2  |
| 3. Database Design .....                                                   | 3  |
| 3.1. ER Diagram .....                                                      | 3  |
| 3.2. Tables & Attributes .....                                             | 4  |
| 3.3. Relationship Sets .....                                               | 7  |
| 3.4. Constraints & Indexes .....                                           | 7  |
| 3.5. Views .....                                                           | 8  |
| 3.6. Stored Function .....                                                 | 9  |
| 4. Main Functionalities .....                                              | 10 |
| 4.1. Student .....                                                         | 10 |
| 4.2. Organizer .....                                                       | 10 |
| 4.3. Administrator .....                                                   | 10 |
| 5. Interface Pages .....                                                   | 11 |
| 6. SQL Query Implementation .....                                          | 17 |
| 6.1. Q1 + Q10: Events List with JOIN and Date Filtering .....              | 17 |
| 6.2. Q2: Ticket Sales Aggregation (via View) .....                         | 18 |
| 6.3. Q3: Remaining Capacity via Correlated Subquery .....                  | 18 |
| 6.4. Q4: Multi-JOIN Ticket History .....                                   | 19 |
| 6.5. Q5: Ticket Purchase (INSERT with Transaction + Stored Function) ..... | 19 |
| 6.6. Q6–Q8: State-Modifying Queries .....                                  | 20 |
| 6.7. Q9: Popular Categories (HAVING) .....                                 | 20 |
| 7. Authentication & Authorization .....                                    | 20 |
| 8. How to Run .....                                                        | 21 |
| 9. Status Against Course Requirements .....                                | 23 |

## 1. Team Roster

---

This report is the final submission for the CampusEvents term project. It is submitted by the two-person team listed below. Per Dr. Tu's course guidelines, two-person groups carry the same deliverable expectations as a three-person group, and the scope of CampusEvents reflects that.

| Name             | Email                  | Role      |
|------------------|------------------------|-----------|
| Trevor Flahardy  | trevorflahardy@usf.edu | Developer |
| Sofia Cobo Navas | scobonavas@usf.edu     | Developer |

## 2. System Overview

---

CampusEvents is a web-based enterprise information system for managing campus events at a university. The platform serves three distinct user roles: **students**, who browse events and register for them; **organizers**, who create and manage them; and **administrators**, who control the entire system — users, events, and categories. The core design goal is a centralized, queryable system that replaces ad-hoc flyers and scattered email announcements.

The system follows a standard three-tier architecture. The **interface tier** is a React 19 single-page application built with Vite, styled with Tailwind CSS v4. The **application tier** is a TypeScript REST API running on the Bun runtime via the Hono framework. The **data tier** is a PostgreSQL 16 relational database, accessed through the `postgres.js` driver using raw parameterized SQL — no ORM is used at any layer, per course requirements.

| Tier        | Technology                        | Port  | Responsibility                                     |
|-------------|-----------------------------------|-------|----------------------------------------------------|
| Interface   | React 19 + Vite + Tailwind CSS v4 | :5173 | Rendering, routing, client-side state              |
| Application | Bun + Hono (TypeScript REST API)  | :3000 | Validation, auth, SQL via <code>postgres.js</code> |
| Data        | PostgreSQL 16 (Docker)            | :5432 | Storage, constraints, referential integrity        |



Figure 1: Three-tier architecture — the React SPA issues `fetch` calls to the Hono REST API, which sends parameterized SQL to PostgreSQL via the `postgres.js` driver.

`postgres.js` serves as the JDBC/ODBC equivalent in this stack: a lightweight PostgreSQL client for JavaScript that sends parameterized SQL directly to the database using tagged template literals (e.g., `sql`SELECT * FROM users WHERE id = ${id}``). All user-supplied values are automatically escaped, preventing SQL injection at the driver level.

## 3. Database Design

### 3.1. ER Diagram

The entity-relationship diagram below represents the conceptual data model for CampusEvents. The physical schema implements five of the six entities directly; the sixth (`images`) is a supporting table for uploaded media storage.

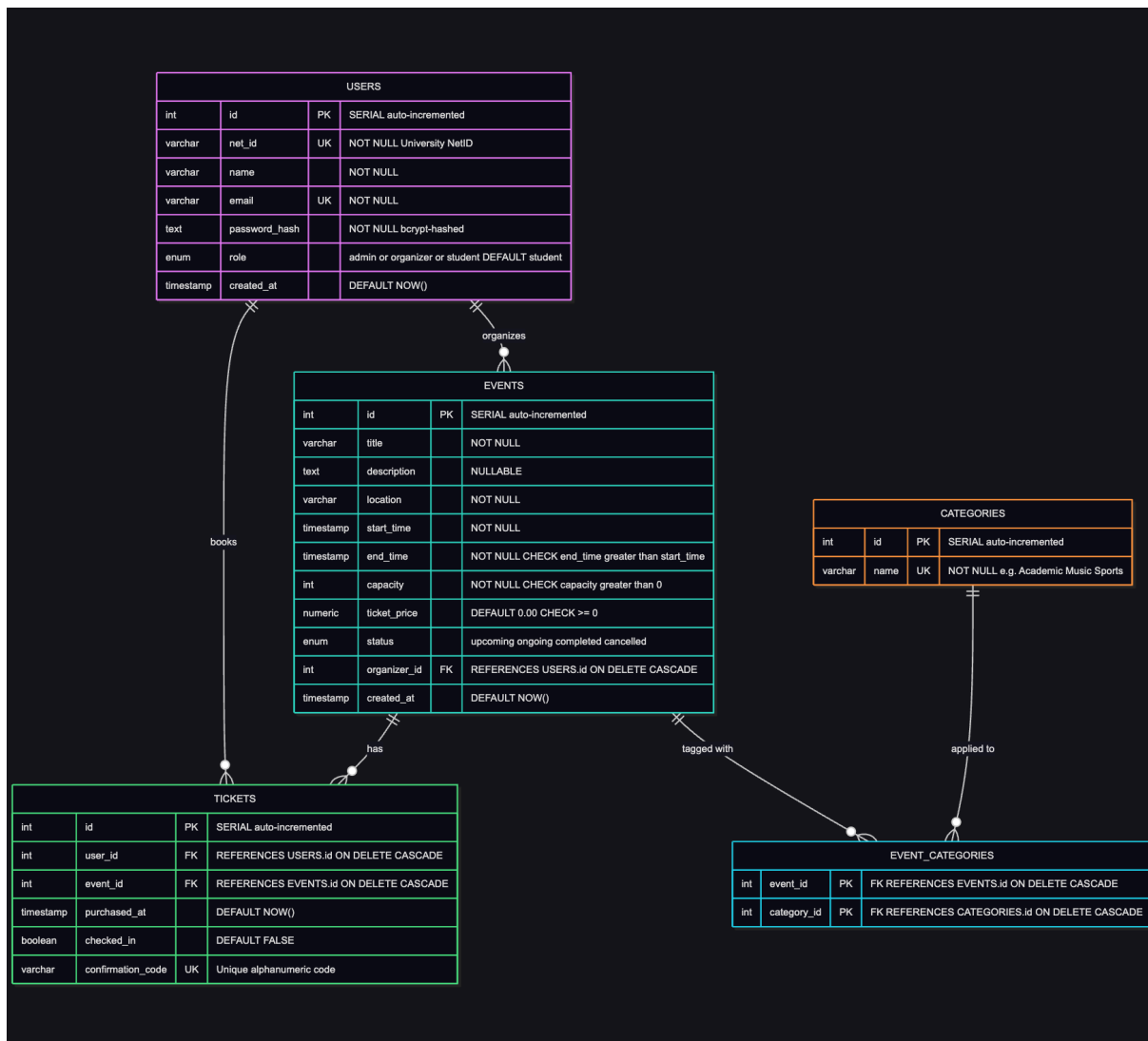


Figure 2: ER diagram for CampusEvents — entity sets, attributes, and cardinality annotations. The `images` table is omitted from the ER diagram as it is a supporting storage relation with no direct business-logic relationships.

### 3.2. Tables & Attributes

**USERS** — Represents every account in the system. The `role` enum determines what the account can do: `student` accounts browse and register; `organizer` accounts create and manage events; `admin` accounts have full system access including role assignment and category management. Passwords are stored exclusively as bcrypt hashes; plaintext is never persisted.

| Attribute | Type   | Constraint  | Notes                          |
|-----------|--------|-------------|--------------------------------|
| <u>id</u> | SERIAL | PRIMARY KEY | Auto-incremented surrogate key |

| Attribute     | Type         | Constraint        | Notes                               |
|---------------|--------------|-------------------|-------------------------------------|
| net_id        | VARCHAR(50)  | NOT NULL, UNIQUE  | University NetID (e.g., jsmith22)   |
| name          | VARCHAR(100) | NOT NULL          | Full display name                   |
| email         | VARCHAR(150) | NOT NULL, UNIQUE  | University email address            |
| password_hash | TEXT         | NOT NULL          | bcrypt hash; plaintext never stored |
| role          | user_role    | DEFAULT 'student' | admin   organizer   student         |
| profile_photo | TEXT         | nullable          | URL path to uploaded avatar         |
| created_at    | TIMESTAMP    | DEFAULT NOW()     | Account creation time-stamp         |

**EVENTS** — The central entity of the system. Every event has a designated organizer (FK to USERS), a time range, a physical venue, and a capacity ceiling. The `status` enum drives filtering across the UI; its value is computed at query time from the event's dates rather than stored statically, so completed events update automatically without a background job. Two additional fields, `latitude` and `longitude`, store GPS coordinates for the Mapbox map preview.

| Attribute    | Type          | Constraint                               | Notes                                                |
|--------------|---------------|------------------------------------------|------------------------------------------------------|
| <u>id</u>    | SERIAL        | PRIMARY KEY                              | Auto-incremented surrogate key                       |
| title        | VARCHAR(200)  | NOT NULL                                 | Event display name                                   |
| description  | TEXT          | nullable                                 | Optional long-form description                       |
| location     | VARCHAR(200)  | NOT NULL                                 | Venue name or address string                         |
| start_time   | TIMESTAMP     | NOT NULL                                 | Event start date and time                            |
| end_time     | TIMESTAMP     | NOT NULL                                 | CHECK enforces <code>end_time &gt; start_time</code> |
| capacity     | INTEGER       | CHECK > 0                                | Maximum attendee count                               |
| ticket_price | NUMERIC(10,2) | DEFAULT 0.00, CHECK <code>&gt;= 0</code> | 0.00 denotes a free event                            |
| status       | event_status  | DEFAULT 'upcoming'                       | upcoming   ongoing   completed   cancelled           |

| Attribute    | Type             | Constraint    | Notes                             |
|--------------|------------------|---------------|-----------------------------------|
| organizer_id | INTEGER          | FK → USERS.id | References the event creator      |
| banner_url   | TEXT             | nullable      | URL path to uploaded banner image |
| latitude     | DOUBLE PRECISION | nullable      | GPS latitude for map pin          |
| longitude    | DOUBLE PRECISION | nullable      | GPS longitude for map pin         |
| created_at   | TIMESTAMP        | DEFAULT NOW() | Record insertion time-stamp       |

**TICKETS** — A ticket represents a confirmed booking between a user and an event. The composite `UNIQUE(user_id, event_id)` constraint enforces the one-ticket-per-user-per-event business rule at the schema level — if a student tries to purchase a second ticket for the same event, PostgreSQL raises error 23505 and the API returns 409 Conflict. Each ticket carries a short alphanumeric `confirmation_code` displayed to the user and used for check-in.

| Attribute              | Type        | Constraint       | Notes                           |
|------------------------|-------------|------------------|---------------------------------|
| id                     | SERIAL      | PRIMARY KEY      | Auto-incremented surrogate key  |
| user_id                | INTEGER     | FK → USERS.id    | References the ticket holder    |
| event_id               | INTEGER     | FK → EVENTS.id   | References the booked event     |
| purchased_at           | TIMESTAMP   | DEFAULT NOW()    | Booking timestamp               |
| checked_in             | BOOLEAN     | DEFAULT FALSE    | Toggled TRUE on event entry     |
| confirmation_code      | VARCHAR(20) | UNIQUE           | Short alphanumeric booking code |
| ( user_id , event_id ) | —           | UNIQUE composite | Prevents duplicate bookings     |

**CATEGORIES** — A lookup table of event tags. The schema seeds seven default categories (Academic, Music, Sports, Career, Social, Arts, Technology) using `ON CONFLICT DO NOTHING`, making the file idempotent across restarts. Admins may add categories at runtime.

**EVENT\_CATEGORIES** — Resolves the many-to-many relationship between EVENTS and CATEGORIES. The composite primary key `(event_id, category_id)` ensures each pairing is unique, and `ON DELETE CASCADE` on both foreign keys keeps the join table clean automatically.

**IMAGES** — Stores uploaded event banners and user profile photos as base-64 data directly in the database rather than on the file system. This design choice means media survives container restarts, branch switches, and fresh clones without any external file-server configuration.

| Attribute               | Type                   | Constraint                    | Notes                                                   |
|-------------------------|------------------------|-------------------------------|---------------------------------------------------------|
| <code>id</code>         | <code>SERIAL</code>    | <code>PRIMARY KEY</code>      |                                                         |
| <code>filename</code>   | <code>TEXT</code>      | <code>NOT NULL, UNIQUE</code> | Generated name used as the URL key                      |
| <code>mime_type</code>  | <code>TEXT</code>      | <code>NOT NULL</code>         | <code>image/jpeg</code> , <code>image/png</code> , etc. |
| <code>data</code>       | <code>TEXT</code>      | <code>NOT NULL</code>         | Base-64-encoded binary data                             |
| <code>created_at</code> | <code>TIMESTAMP</code> | <code>DEFAULT NOW()</code>    |                                                         |

### 3.3. Relationship Sets

| Entity A | Entity B   | Name               | Cardinality | Description                                                                             |
|----------|------------|--------------------|-------------|-----------------------------------------------------------------------------------------|
| USERS    | EVENTS     | <i>organizes</i>   | 1 : N       | An organizer may create many events; each event has exactly one organizer               |
| USERS    | TICKETS    | <i>books</i>       | 1 : N       | A student may hold tickets to many events; each ticket belongs to one user              |
| EVENTS   | TICKETS    | <i>has</i>         | 1 : N       | An event may have many tickets; each ticket corresponds to exactly one event            |
| EVENTS   | CATEGORIES | <i>tagged with</i> | M : N       | Resolved via <code>EVENT_CATEGORIES</code> ; an event may belong to multiple categories |

### 3.4. Constraints & Indexes

The following business rules are enforced at the schema level, independent of application behavior:

- `CHECK (end_time > start_time)` — Prevents events from being inserted with an inverted time range.
- `CHECK (capacity > 0)` — Every event must allow at least one attendee.
- `CHECK (ticket_price >= 0)` — Prevents negative ticket prices.
- `UNIQUE (user_id, event_id)` on `TICKETS` — One ticket per user per event; violation raises SQL state 23505.
- `UNIQUE (organizer_id, title, start_time)` on `EVENTS` — Prevents duplicate events from the same organizer at the same time; also the target of seed data's `ON CONFLICT` clause.
- `ON DELETE CASCADE` on `TICKETS` and `EVENT_CATEGORIES` — Deleting a user or event automatically removes all dependent ticket and category-link rows.

Five indexes support the most common query patterns:

| Index                              | Column(s)                         | Query it accelerates                                |
|------------------------------------|-----------------------------------|-----------------------------------------------------|
| <code>idx_events_status</code>     | <code>events(status)</code>       | Filter upcoming / ongoing events on every page load |
| <code>idx_events_organizer</code>  | <code>events(organizer_id)</code> | Organizer dashboard event list                      |
| <code>idx_events_start_time</code> | <code>events(start_time)</code>   | Date-range filtering (Q10 <code>BETWEEN</code> )    |
| <code>idx_tickets_user</code>      | <code>tickets(user_id)</code>     | My Tickets page — Q4 multi-JOIN                     |
| <code>idx_tickets_event</code>     | <code>tickets(event_id)</code>    | Attendee list per event, capacity sub-query         |

### 3.5. Views

Two SQL views are defined in `schema.sql` using `CREATE OR REPLACE VIEW`, making them idempotent across every backend restart. They provide role-scoped projections of the underlying tables — a public-facing browse surface and an organizer-facing aggregation.

**v\_upcoming\_events** — Joins `EVENTS` with `USERS` and pre-filters to `status = 'upcoming'`. Exposes organizer name and email alongside event fields while hiding internal columns such as `organizer_id` and `created_at` that are irrelevant to a browsing student.

**v\_event\_stats** — Aggregates ticket counts per event using `GROUP BY` and `COUNT(*)` over the `EVENTS-TICKETS` join. This view is queried directly by `GET /api/events/stats`, which powers the organizer dashboard's stat cards.

```
-- v_upcoming_events: public browse projection (student-facing)
CREATE OR REPLACE VIEW v_upcoming_events AS
  SELECT e.id, e.title, e.description, e.location,
         e.start_time, e.end_time, e.capacity, e.ticket_price,
         e.status, e.banner_url, e.latitude, e.longitude,
         u.name AS organizer_name,
         u.email AS organizer_email
  FROM events e
  INNER JOIN users u ON e.organizer_id = u.id
  WHERE e.status = 'upcoming'
  ORDER BY e.start_time;

-- v_event_stats: organizer ticket-sales summary (organizer-facing)
CREATE OR REPLACE VIEW v_event_stats AS
  SELECT e.id AS event_id,
         e.title, e.status, e.capacity,
         e.ticket_price, e.end_time, e.organizer_id,
         u.name AS organizer_name,
         COUNT(t.id)::int AS tickets_sold,
         (e.capacity - COUNT(t.id)::int) AS spots_remaining
  FROM events e
  INNER JOIN users u ON e.organizer_id = u.id
```



```
LEFT JOIN tickets t ON e.id = t.event_id
GROUP BY e.id, e.title, e.status, e.capacity,
         e.ticket_price, e.end_time, e.organizer_id, u.name;
```

The `GET /api/events/stats` route queries `v_event_stats` directly:

```
// routes/events.ts --- stats endpoint backed by the view
const rows = await sql`
  SELECT event_id, title, status, capacity, tickets_sold, spots_remaining
  FROM v_event_stats
  WHERE status != 'cancelled' AND end_time >= ${now}
`;
```

### 3.6. Stored Function

`check_event_capacity(p_event_id INTEGER)` is a PL/pgSQL function that returns a single row containing the event's capacity, current ticket count, spots remaining, and an `is_available` boolean. It is declared `STABLE` because it reads but does not modify data. The function is called inside the ticket purchase transaction — after the `SELECT ... FOR UPDATE` row lock is acquired — to atomically determine whether the event can still accept a registration.

```
CREATE OR REPLACE FUNCTION check_event_capacity(p_event_id INTEGER)
RETURNS TABLE (
  event_id    INTEGER,
  title       VARCHAR,
  capacity    INTEGER,
  tickets_sold BIGINT,
  spots_remaining BIGINT,
  is_available BOOLEAN
) LANGUAGE plpgsql STABLE AS $$
BEGIN
  RETURN QUERY
    SELECT e.id, e.title, e.capacity,
           COUNT(t.id),
           (e.capacity - COUNT(t.id)),
           (e.capacity - COUNT(t.id)) > 0
    FROM events e
   LEFT JOIN tickets t ON e.id = t.event_id
  WHERE e.id = p_event_id
  GROUP BY e.id, e.title, e.capacity;
END;
$;
```

## 4. Main Functionalities

---

### 4.1. Student

Students are the primary users of the platform. After registering with a university email and NetID, a student can:

- **Browse events** — Search by keyword (case-insensitive `ILIKE`), filter by category, filter by date range, and sort by date or title. All event statuses are visible.
- **View event detail** — See the full event description, venue, time range, ticket price, and real-time remaining capacity computed by `check_event_capacity`.
- **Book a ticket** — Click “Book Ticket” on any upcoming event with capacity remaining. The backend issues a `SELECT ... FOR UPDATE` + `INSERT` transaction and returns a unique confirmation code.
- **Manage bookings** — The My Tickets page lists all bookings with confirmation codes and check-in status. Students can cancel a ticket or self-check-in during the 30-minute window before the event starts through its end time.
- **Edit profile** — Update display name, email, and upload a profile photo stored in the `images` table.

### 4.2. Organizer

Organizers create and manage events. The Organizer Dashboard provides:

- **Stats overview** — Stat cards show total events created, upcoming event count, and total attendees checked in, drawn from `v_event_stats`.
- **Create an event** — A full creation form collects title, description, location, time range, capacity, ticket price, and one or more categories. An optional Mapbox GL JS map picker lets the organizer drop a GPS pin, which renders as an interactive map preview on the event detail page. A banner image can be uploaded.
- **Edit events** — Any field of an owned event can be updated; changes issue a dynamic `UPDATE` with only the changed fields.
- **Cancel an event** — Sets `status = 'cancelled'` via a targeted `UPDATE`.
- **View attendees** — The attendee panel lists all ticket holders with name, email, check-in status, and confirmation code.

### 4.3. Administrator

Admins have unrestricted access via the Admin Panel:

- **User management** — View all registered accounts. Promote or demote any user’s role in real time via an inline dropdown (issues `PATCH /api/users/:id/role`).
- **Event management** — Full CRUD over all events in the system, regardless of organizer ownership.
- **Category management** — Create and delete event categories.
- **All organizer capabilities** — Admins can create and manage events on behalf of any organizer.

## 5. Interface Pages

The frontend ships eight routed pages, well above the three-page minimum. Routing is managed by `react-router-dom` v7; protected routes redirect unauthenticated users to `/login`. The UI uses a glassmorphism design system with a deep-green brand color, and supports full light and dark mode.

| Page<br>(in <code>frontend/src/pages/</code> ) | Route                    | Purpose                                                                     |
|------------------------------------------------|--------------------------|-----------------------------------------------------------------------------|
| <code>Landing.tsx</code>                       | <code>/</code>           | Public landing page with live event count and feature summary               |
| <code>Login.tsx</code>                         | <code>/login</code>      | JWT-based sign-in form                                                      |
| <code>Register.tsx</code>                      | <code>/register</code>   | Account creation; role defaults to <code>student</code>                     |
| <code>BrowseEvents.tsx</code>                  | <code>/events</code>     | Event discovery: keyword search, category filter, date range, status filter |
| <code>EventDetail.tsx</code>                   | <code>/events/:id</code> | Full event info, real-time remaining capacity, Book Ticket action           |
| <code>MyTickets.tsx</code>                     | <code>/my-tickets</code> | Student's active bookings with confirmation codes and check-in control      |
| <code>OrganizerDashboard.tsx</code>            | <code>/dashboard</code>  | Create, edit, and cancel events; attendee list; ticket stats                |
| <code>AdminPanel.tsx</code>                    | <code>/admin</code>      | System management: users, events, categories, role assignment               |

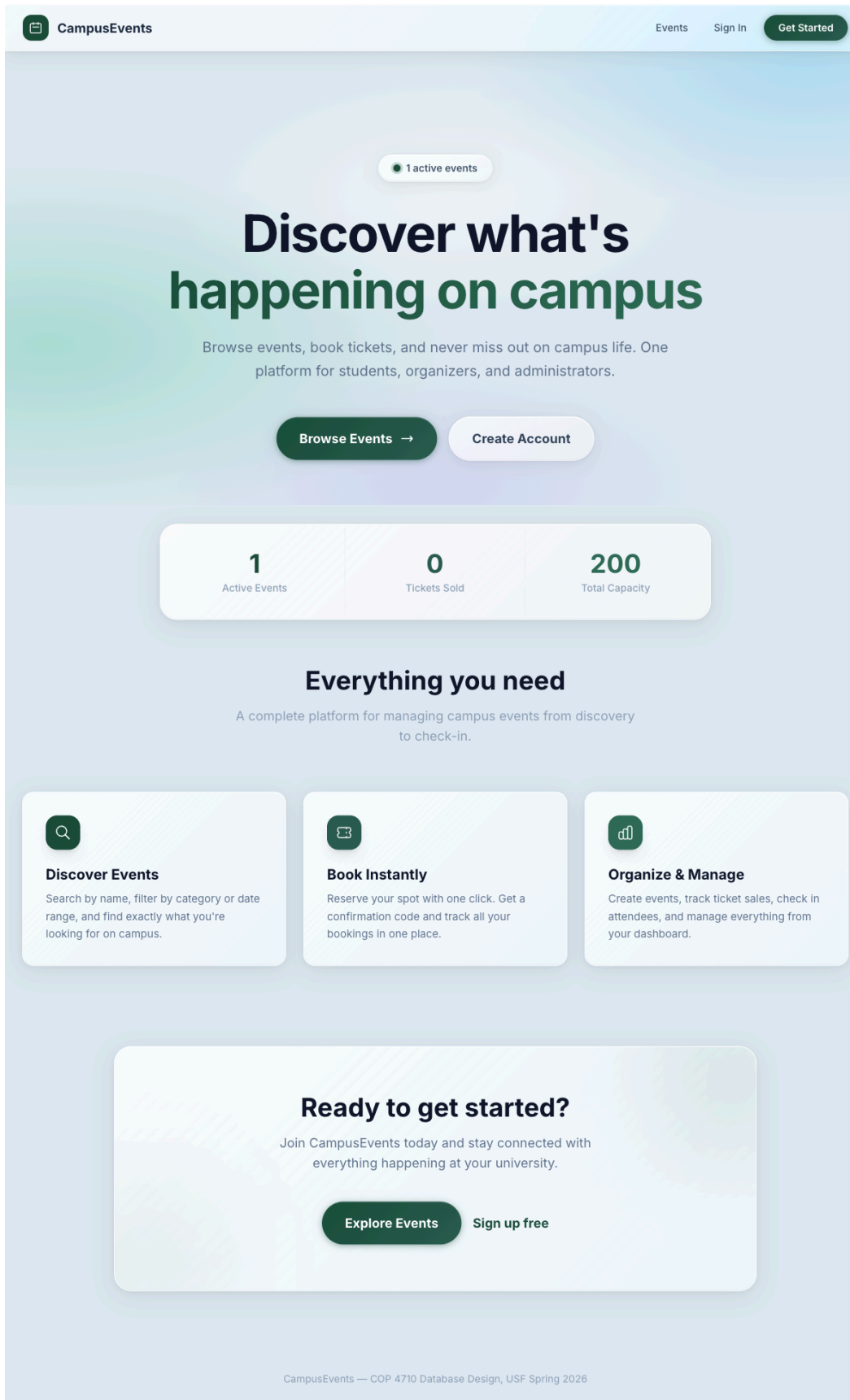


Figure 3: **Landing (/)** — public hero section. The active-event count and total capacity figures are fetched live from `GET /api/events` on every page load, pulling real data from the database.

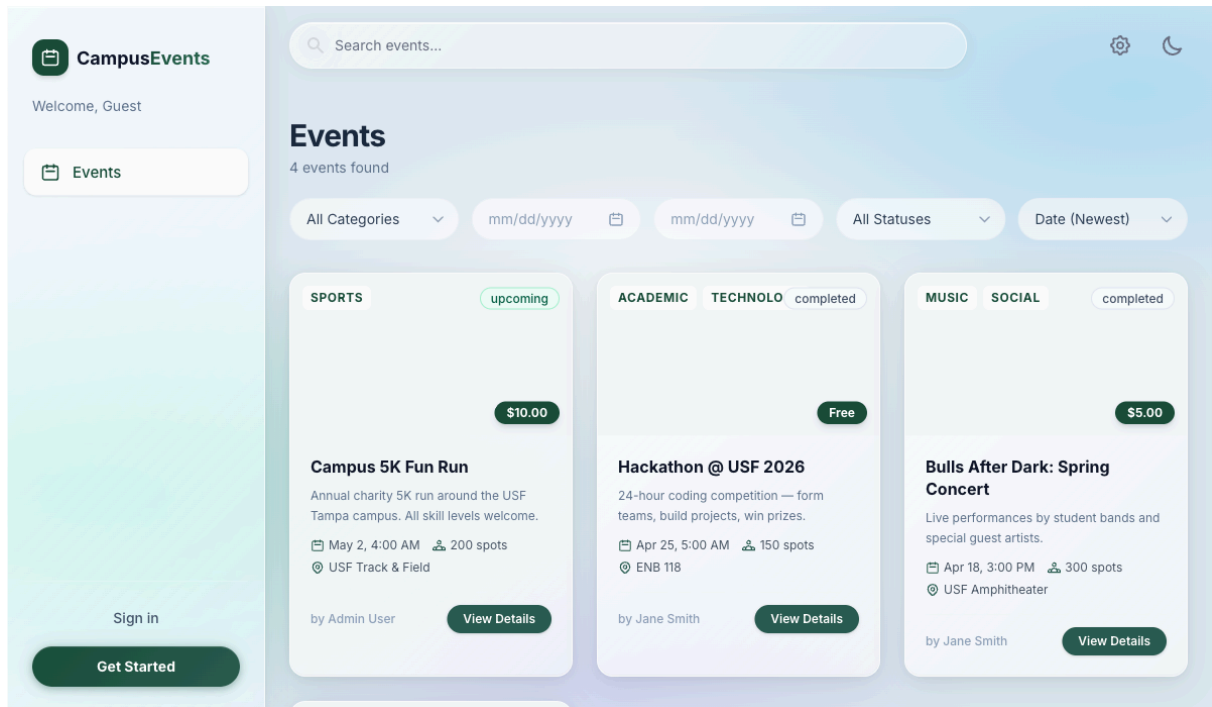


Figure 4: **Browse Events** (`/events`) — event grid backed by `GET /api/events`. Keyword search uses `ILIKE`, category filter JOINS through `EVENT_CATEGORIES`, date range uses `BETWEEN`, and status filter applies a `WHERE` clause — all composed server-side.

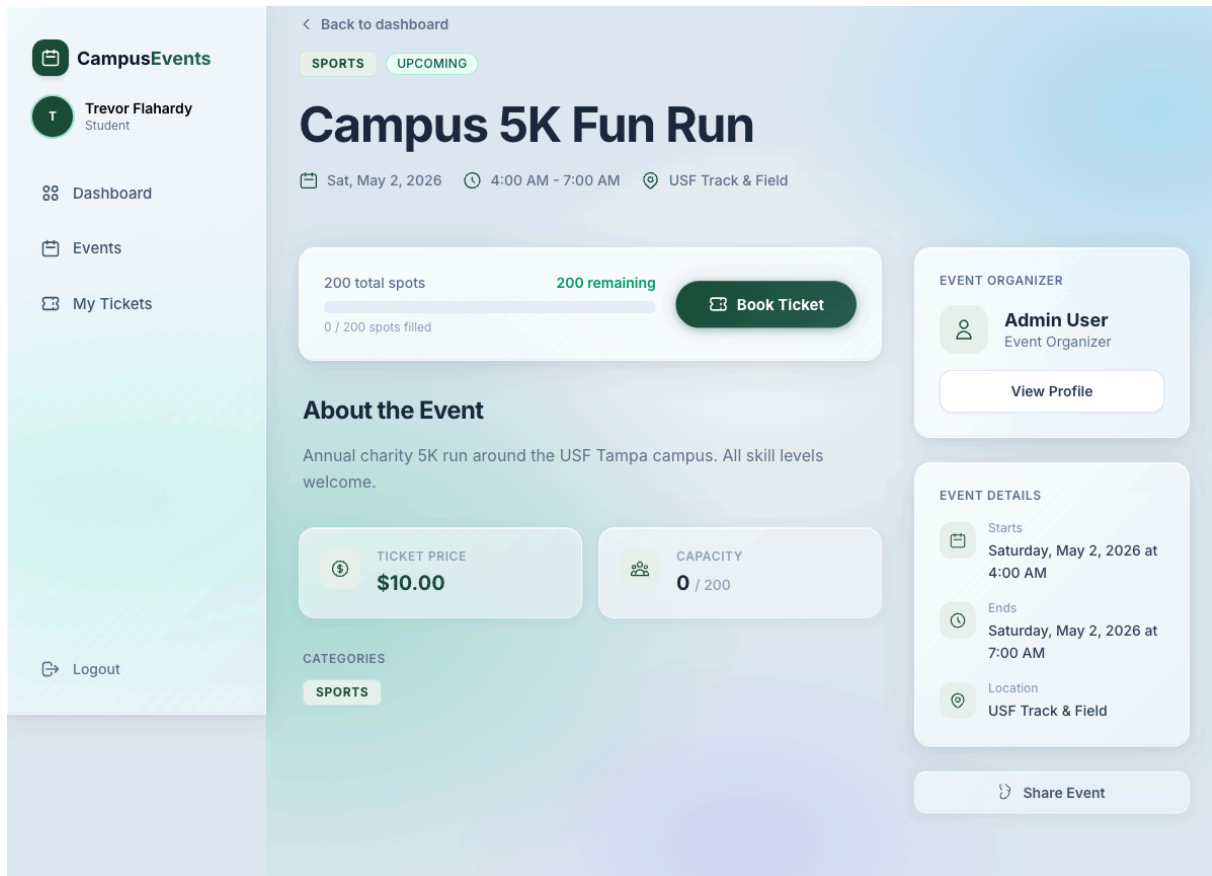


Figure 5: **Event Detail** ( `/events/:id` ) — full event info with a real-time remaining-capacity counter. Capacity is computed by the `check_event_capacity` stored function. The Book Ticket button triggers the `SELECT ... FOR UPDATE + INSERT` transaction.

**CampusEvents**

**Trevor Flahardy**  
Student

Dashboard

Events

**My Tickets**

Logout

## My Tickets

View and manage your upcoming campus experiences. Your digital entry passes for the season.

**CAREER** completed

### Spring Career Fair 2026

Marshall Student Center Ballroom

Event ended

| DATE         | TIME    | PRICE | CHECK-IN |
|--------------|---------|-------|----------|
| Apr 10, 2026 | 6:00 AM | Free  | No       |

VERIFICATION

CONF. NUMBER  
**CF26A001**

[View Event](#)

**ACADEMIC TECHNOLOGY** completed

### Hackathon @ USF 2026

ENB 118

Event ended

| DATE         | TIME    | PRICE | CHECK-IN |
|--------------|---------|-------|----------|
| Apr 25, 2026 | 5:00 AM | Free  | No       |

VERIFICATION

CONF. NUMBER  
**HACK2601**

[View Event](#)

Showing 2 active tickets

Figure 6: **My Tickets** ( /my-tickets ) — authenticated student’s bookings. Each card shows the unique confirmation code, check-in status, and full event info joined across TICKETS, EVENTS, and CATEGORIES.

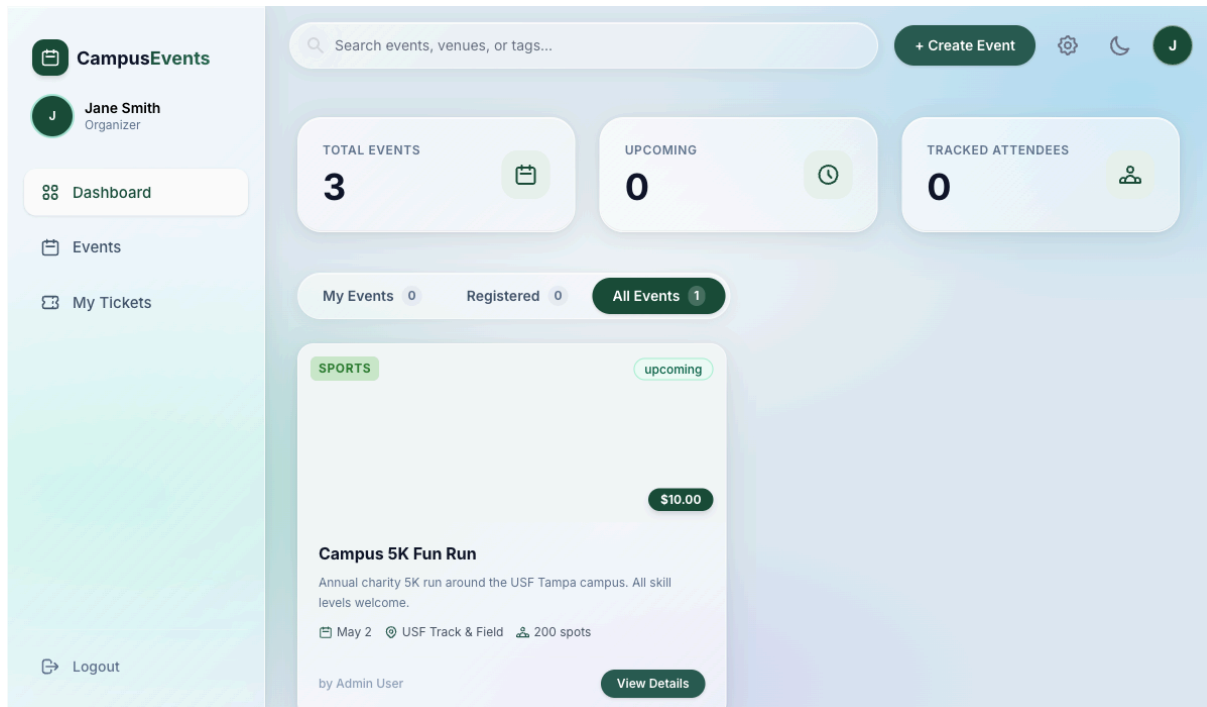


Figure 7: **Organizer Dashboard** (`/dashboard`) — stat cards and event grid. Ticket totals come from the `v_event_stats` view. The All Events tab shows every event in the system; the “+ Create Event” button opens a full creation form with an optional Mapbox map picker.

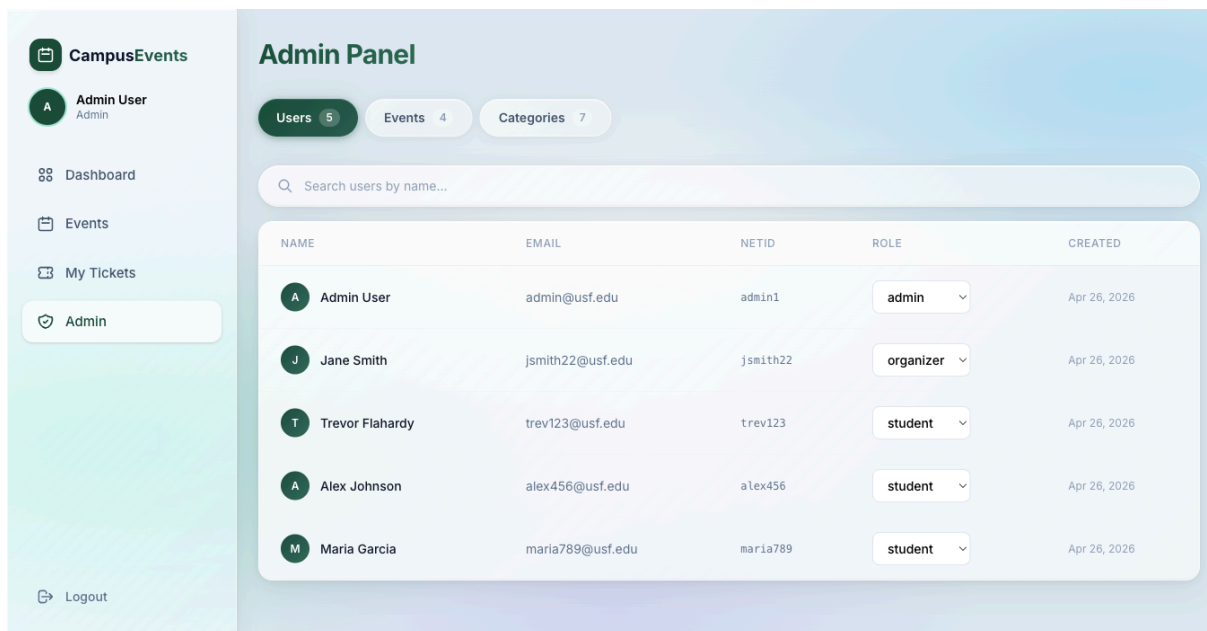


Figure 8: **Admin Panel** (`/admin`) — full user directory with inline role dropdowns. Selecting a new role immediately issues `PATCH /api/users/:id/role`; the change takes effect on the target user’s next login. The Events and Categories tabs provide full system-wide CRUD.



## 6. SQL Query Implementation

All ten distinct SQL query types are implemented as live REST endpoints driving real UI features. Every query is issued through `postgres.js` tagged template literals, which parameterize all user-supplied values automatically. The table below maps each query type to its handler file and the page that exercises it.

| #   | Query Type           | Endpoint (handler)                                  | UI Caller              |
|-----|----------------------|-----------------------------------------------------|------------------------|
| Q1  | SELECT + JOIN        | GET /api/events<br>routes/events.ts                 | BrowseEvents.tsx       |
| Q2  | GROUP BY + COUNT     | GET /api/events/stats<br>routes/events.ts           | OrganizerDashboard.tsx |
| Q3  | SELECT + subquery    | GET /api/events/:id<br>routes/events.ts             | EventDetail.tsx        |
| Q4  | SELECT + multi-JOIN  | GET /api/tickets/user/:id<br>routes/tickets.ts      | MyTickets.tsx          |
| Q5  | INSERT (transaction) | POST /api/tickets<br>routes/tickets.ts              | EventDetail.tsx        |
| Q6  | UPDATE               | PATCH /api/tickets/:id/checkin<br>routes/tickets.ts | MyTickets.tsx          |
| Q7  | UPDATE               | PATCH /api/events/:id<br>routes/events.ts           | OrganizerDashboard.tsx |
| Q8  | DELETE               | DELETE /api/tickets/:id<br>routes/tickets.ts        | MyTickets.tsx          |
| Q9  | SELECT + HAVING      | GET /api/categories/popular<br>routes/categories.ts | BrowseEvents.tsx       |
| Q10 | SELECT + BETWEEN     | GET /api/events?from=&to=<br>routes/events.ts       | BrowseEvents.tsx       |

### 6.1. Q1 + Q10: Events List with JOIN and Date Filtering

The browse endpoint dynamically composes a `WHERE` clause from optional query parameters. Each condition is a safe tagged-template fragment; they are reduced with `AND` to form the full predicate.

```
// routes/events.ts --- GET /api/events
const conditions = [sql`TRUE`];
// Q10: BETWEEN-style date filtering via >= / <=
if (from) conditions.push(sql`e.start_time >= ${new Date(from)}`);
if (to) conditions.push(sql`e.start_time <= ${new Date(to)}`);
```

```

if (search) conditions.push(sql`e.title ILIKE ${"%" + search + "%"}`);
const where = conditions.reduce((a, c) => sql`${a} AND ${c}`);

// Q1: SELECT with JOIN --- events with organizer name
const rows = await sql`
  SELECT e.id, e.title, e.description, e.location,
         e.start_time, e.end_time, e.capacity, e.ticket_price,
         e.status, e.banner_url, e.latitude, e.longitude,
         u.name AS organizer_name
  FROM events e
  INNER JOIN users u ON e.organizer_id = u.id
  WHERE ${where}
  ORDER BY e.start_time
`;

```

When a category filter is active, a second `INNER JOIN` through `event_categories` restricts results to that category.

### 6.2. Q2: Ticket Sales Aggregation (via View)

The stats endpoint queries `v_event_stats`, which pre-aggregates `tickets_sold` and `spots_remaining` using `GROUP BY + COUNT` on the underlying `EVENTS-TICKETS` join. Using the view keeps the route lean and demonstrates the database-side view bonus feature.

```

// routes/events.ts --- GET /api/events/stats
const rows = await sql`
  SELECT event_id, title, status, capacity, tickets_sold, spots_remaining
  FROM v_event_stats
  WHERE status != 'cancelled' AND end_time >= ${now}
`;

```

### 6.3. Q3: Remaining Capacity via Correlated Subquery

The event detail endpoint computes `spots_remaining` inline using a correlated subquery against the `TICKETS` table. This gives an accurate real-time count without a separate query round-trip.

```

// routes/events.ts --- GET /api/events/:id
const rows = await sql`
  SELECT e.id, e.title, e.description, e.location,
         e.start_time, e.end_time, e.capacity, e.ticket_price,
         e.status, e.banner_url, e.latitude, e.longitude,
         u.name AS organizer_name,
         (e.capacity - (SELECT COUNT(*)::int FROM tickets WHERE event_id = e.id))
         AS spots_remaining
  FROM events e
  INNER JOIN users u ON e.organizer_id = u.id
  WHERE e.id = ${id}
`;

```

#### 6.4. Q4: Multi-JOIN Ticket History

The My Tickets endpoint joins TICKETS with EVENTS in one query, then batch-fetches all associated categories in a second query using `ANY(array)` to avoid an N+1 pattern. Results are merged in application code before returning.

```
// routes/tickets.ts --- GET /api/tickets/user/:userId
const rows = await sql`
  SELECT t.id AS ticket_id, t.purchased_at, t.checked_in, t.confirmation_code,
         e.id AS event_id, e.title AS event_title, e.location AS event_location,
         e.start_time AS event_start_time, e.end_time AS event_end_time,
         e.status AS event_status, e.ticket_price
  FROM tickets t
  INNER JOIN events e ON t.event_id = e.id
  WHERE t.user_id = ${userId}
  ORDER BY e.start_time
`;

// Batch-fetch categories for all event IDs in one shot (avoids N+1)
const categoryRows = await sql`
  SELECT ec.event_id, c.id, c.name
  FROM categories c
  INNER JOIN event_categories ec ON c.id = ec.category_id
  WHERE ec.event_id = ANY(${eventIds})
`;
```

#### 6.5. Q5: Ticket Purchase (INSERT with Transaction + Stored Function)

The ticket purchase flow wraps the capacity check and INSERT in a single database transaction with a row-level lock on the event. The `SELECT ... FOR UPDATE` blocks any concurrent purchaser until this transaction commits, so the ticket count read by `check_event_capacity` is authoritative at insert time — the mechanism that actually prevents oversell under concurrent requests.

```
// routes/tickets.ts --- POST /api/tickets
const outcome = await sql.begin(async (tx) => {
  // Lock the event row to serialize concurrent purchase attempts
  const [event] = await tx`
    SELECT id, capacity, status FROM events WHERE id = ${eventId} FOR UPDATE
  `;
  if (!event) return { kind: "not_found" };
  if (event.status === "cancelled") return { kind: "cancelled" };

  // Stored function checks current capacity within the held lock
  const [cap] = await tx`SELECT * FROM check_event_capacity(${eventId})`;
  if (!cap || !cap.isAvailable) return { kind: "sold_out" };

  // Q5: INSERT new ticket and return the created row
  const [inserted] = await tx`
    INSERT INTO tickets (user_id, event_id, confirmation_code)
    VALUES (${userId}, ${eventId}, ${confirmationCode})
  `;
```

```

    RETURNING *
  `;
  return { kind: "ok", ticket: inserted };
});

```

## 6.6. Q6–Q8: State-Modifying Queries

```

// Q6: UPDATE --- check in a ticket (toggle boolean flag)
const [updated] = await sql`
  UPDATE tickets SET checked_in = TRUE WHERE id = ${id} RETURNING *
`;

// Q7: UPDATE --- cancel or edit an event (dynamic SET clause)
// sql() helper maps a plain object's keys to SQL column names safely
const [updated] = await sql`
  UPDATE events
  SET ${sql(updates, ...Object.keys(updates))}
  WHERE id = ${id}
  RETURNING *
`;

// Q8: DELETE --- cancel (remove) a ticket
const deleted = await sql`DELETE FROM tickets WHERE id = ${id} RETURNING *`;

```

## 6.7. Q9: Popular Categories (HAVING)

```

// routes/categories.ts --- GET /api/categories/popular
const rows = await sql`
  SELECT c.id AS category_id, c.name,
         COUNT(ec.event_id)::int AS event_count
  FROM categories c
  INNER JOIN event_categories ec ON c.id = ec.category_id
  GROUP BY c.id, c.name
  HAVING COUNT(ec.event_id) > 1
`;

```

## 7. Authentication & Authorization

User accounts with userID/password are implemented as bonus feature 1. Passwords are hashed with `bcrypt` ( `@node-rs/bcrypt` , cost factor 10) on registration and verified on login. A successful login returns a JWT signed with `HS256` containing the user's `id` and `role` . The token is stored in `localStorage` on the client and attached to subsequent requests via `Authorization: Bearer` .

All protected routes pass through `authMiddleware` at `backend/src/middleware/auth.ts`, which verifies the token and injects `userId` and `userRole` into the request context. A `requireRole(...)` factory then gates individual routes to specific roles.

```
// middleware/auth.ts
export async function authMiddleware(c: Context<AuthEnv>, next: Next) {
  const header = c.req.header("Authorization");
  if (!header?.startsWith("Bearer "))
    return c.json({ error: "Missing or invalid Authorization header" }, 401);
  const payload = await verify(header.slice(7), JWT_SECRET, "HS256");
  c.set("userId", payload.sub as number);
  c.set("userRole", payload.role as string);
  await next();
}

export function requireRole(...roles: string[]) {
  return async (c: Context<AuthEnv>, next: Next) => {
    if (!roles.includes(c.get("userRole")))
      return c.json({ error: "Forbidden" }, 403);
    await next();
  };
}
```

Role enforcement is applied at the route level: `POST /api/events` requires `organizer` or `admin`; `PATCH /api/users/:id/role` and `GET /api/users` require `admin`; organizers may only modify their own events (ownership verified with `SELECT WHERE organizer_id = $userId` before any mutation). Every write endpoint additionally validates its request body with a Zod schema before any SQL executes, so the database never receives malformed input.

## 8. How to Run

The entire stack boots with one command from the project root:

```
docker compose up
```

Docker Compose starts three containers — `campusevents_db` (PostgreSQL 16), `campusevents_api` (Bun + Hono), and `campusevents_web` (React + Vite). On startup, the backend applies `schema.sql` via `postgres.js`'s `unsafe()` method, creating all six tables, five indexes, two views, and the stored function. All DDL statements use `IF NOT EXISTS` or `CREATE OR REPLACE`, making the migration idempotent and safe to run on every boot. The backend logs  `Schema applied` when ready; Vite logs the local URL when the bundle is served.

```
# Load demo users, events, and tickets
cd backend && bun run db:seed

# Open the app
open http://localhost:5173

# Backend health check
curl http://localhost:3000/health # → {"status":"ok"}
```

| Email             | Password    | Role                                      |
|-------------------|-------------|-------------------------------------------|
| admin\@usf.edu    | password123 | Administrator — full system access        |
| jsmith22\@usf.edu | password123 | Organizer — event creation and management |
| trev123\@usf.edu  | password123 | Student — browse, register, check in      |

```
futur@TrevMacbook project_4 % docker compose up --build --watch
=> [frontend] exporting to image                                0.0s
=> => exporting layers                                          0.0s
[+] Running 5/2 image sha256:7c42656409a6f734006e1620c196759fc819b0dddeddb17d13a52c408df29e2d 0.0s
✓ Service backend Built 2.9s
✓ Service frontend Built 1.0s
✓ Container campusevents_db Created 0.0s
✓ Container campusevents_api Created 0.0s
✓ Container campusevents_web Created 0.0s
Attaching to campusevents_api, campusevents_db, campusevents_web
● Watch enabled
campusevents_db | PostgreSQL Database directory appears to contain a database; Skipping initialization
campusevents_db | 2026-04-09 19:06:01.122 UTC [1] LOG: starting PostgreSQL 16.13 on aarch64-unknown-linux-musl, compiled by gcc (Alpine 1
.0) 15.2.0, 64-bit
campusevents_db | 2026-04-09 19:06:01.122 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
campusevents_db | 2026-04-09 19:06:01.123 UTC [1] LOG: listening on IPv6 address "::", port 5432
campusevents_db | 2026-04-09 19:06:01.126 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
campusevents_db | 2026-04-09 19:06:01.133 UTC [29] LOG: database system was interrupted; last known up at 2026-04-08 12:58:17 UTC
campusevents_api | $ bun run src/index.ts
campusevents_db | 2026-04-09 19:06:01.192 UTC [29] LOG: database system was not properly shut down; automatic recovery in progress
campusevents_db | 2026-04-09 19:06:01.196 UTC [29] LOG: redo starts at 0/1EFC690
campusevents_db | 2026-04-09 19:06:01.199 UTC [29] LOG: invalid record length at 0/1EFC70: expected at least 24, got 0
campusevents_db | 2026-04-09 19:06:01.199 UTC [29] LOG: redo done at 0/1EFC38 system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed:
0 s
campusevents_db | 2026-04-09 19:06:01.202 UTC [27] LOG: checkpoint starting: end-of-recovery immediate wait
campusevents_db | 2026-04-09 19:06:01.211 UTC [27] LOG: checkpoint complete: wrote 4 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 re
led; write=0.005 s, sync=0.002 s, total=0.009 s; sync files=3, longest=0.001 s, average=0.001 s; distance=1 kB, estimate=1 kB; lsn=0/1EFC70
edo lsn=0/1EFC70
campusevents_db | 2026-04-09 19:06:01.221 UTC [1] LOG: database system is ready to accept connections
campusevents_web | $ vite --host "0.0.0.0"
campusevents_api | {
campusevents_api |   severity_local: "NOTICE",
campusevents_api |   severity: "NOTICE",
campusevents_api |   code: "42P06",
campusevents_api |   message: "schema \"drizzle\" already exists, skipping",
campusevents_api |   file: "schemacmds.c",
campusevents_api |   line: "132",
campusevents_api |   routine: "CreateSchemaCommand",
campusevents_api | }
campusevents_api | {
campusevents_api |   severity_local: "NOTICE",
campusevents_api |   severity: "NOTICE",
```

Figure 9: docker compose up — all three tiers starting in sequence. The API logs “Schema applied” before accepting traffic; Vite serves the React bundle on port 5173. The seed script can be run immediately after to populate demo data.

```

o futur@TrevMacbook project_4 % docker exec -it campusevents_db psql -U campus -d campusevents
psql (16.13)
Type "help" for help.

campusevents=# \dt
               List of relations
 Schema |      Name      | Type  | Owner
-----+-----+-----+-----
 public | categories     | table | campus
 public | event_categories | table | campus
 public | events         | table | campus
 public | images         | table | campus
 public | tickets        | table | campus
 public | users          | table | campus
(6 rows)

campusevents=#

```

Figure 10: Output of `\dt` in `psql` against the running container — six live relations created by `schema.sql`. Running `\dv` additionally shows `v_event_stats` and `v_upcoming_events`; `\df` shows `check_event_capacity`.

## 9. Status Against Course Requirements

The table below maps every graded requirement and bonus item from the assignment handout to our implementation status.

| # | Requirement                                                         | Status                                               |
|---|---------------------------------------------------------------------|------------------------------------------------------|
| 1 | DBMS-backed database with $\geq 3$ relations loaded with data       | ✓ 6 tables + seed                                    |
| 2 | $\geq 8$ distinct SQL query types including state-modifying queries | ✓ 10 implemented                                     |
| 3 | Web-based interface with $\geq 3$ distinct pages                    | ✓ 8 pages                                            |
| 4 | JDBC/ODBC-style connection to send queries to the database          | ✓ <code>postgres.js</code> raw SQL                   |
| — | <b>Bonus</b> — user accounts with <code>userID/password</code>      | ✓ <code>bcrypt</code> + JWT                          |
| — | <b>Bonus</b> — views with per-role projections                      | ✓ 2 views in <code>schema.sql</code>                 |
| — | <b>Bonus</b> — stored procedures/functions on the database side     | ✓ <code>check_event_capacity()</code>                |
| — | <b>Bonus</b> — client-side scripts for application logic            | ✓ React 19 SPA                                       |
| — | <b>Bonus</b> — additional relevant features                         | ✓ transactions, image upload, Mapbox maps, dark mode |

*All four required features are fully implemented. For the bonus items: user accounts use `bcrypt` password hashing and JWT-based session tokens; two SQL views ( `v_upcoming_events` , `v_event_stats` ) provide role-scoped projections of*

*the underlying tables; the `check_event_capacity` PL/pgSQL function encapsulates capacity logic on the database side and is called from within the ticket purchase transaction; and the React single-page application handles all client-side routing, state management, and UI rendering. Additional features beyond the rubric include a row-level-locked ticket purchase transaction to prevent oversell, base-64 image storage in the database for media persistence, Mapbox GL JS map previews for event venues, and full light/dark mode theming.*